

In this case, `grade` is less than 60, so *both* statements in the `else`'s suite execute.

If you do not indent the second `print`, then it's not in the `else`'s suite. So, that statement *always* executes, possibly creating strange incorrect output:

[lick here to view code image](#)

```
In [15]: grade = 100

In [16]: if grade >= 60:
...:     print('Passed')
...: else:
...:     print('Failed')
...:     print('You must take this course again')
...:
Passed
You must take this course again
```

`if...elif...else` Statement

You can test for many cases using the **`if...elif...else` statement**. The following code displays “A” for grades greater than or equal to 90, “B” for grades in the range 80–89, “C” for grades 70–79, “D” for grades 60–69 and “F” for all other grades. Only the action for the *first* True condition executes. Snippet [18] displays C, because `grade` is 77:

```
In [17]: grade = 77

In [18]: if grade >= 90:
...:     print('A')
...: elif grade >= 80:
...:     print('B')
...: elif grade >= 70:
...:     print('C')
...: elif grade >= 60:
...:     print('D')
...: else:
...:     print('F')
...:
C
```

The first condition—`grade >= 90`—is False, so `print('A')` is skipped. The second condition—`grade >= 80`—also is False, so `print('B')` is skipped. The third condition—`grade >= 70`—is True, so `print('C')` executes. Then all the remaining

code in the `if...elif...else` statement is skipped. An `if...elif...else` is faster than separate `if` statements, because condition testing stops as soon as a condition is `True`.

else Is Optional

The `else` in the `if...elif...else` statement is optional. Including it enables you to handle values that do not satisfy *any* of the conditions. When an `if...elif` statement without an `else` tests a value that does not make any of its conditions `True`, the program does not execute any of the statement's suites—the next statement in sequence after the `if...elif...` statement executes. If you specify the `else`, you must place it *after* the last `elif`; otherwise, a `SyntaxError` occurs.

Logic Errors

The incorrectly indented code segment in snippet [16] is an example of a *nonfatal logic error*. The code executes, but it produces incorrect results. For a *fatal logic error in a script*, an exception occurs (such as a `Zero-DivisionError` from an attempt to divide by 0), so Python displays a traceback, then terminates the script. A *fatal error in interactive mode* terminates only the current snippet—then IPython waits for your next input.

3.5 WHILE STATEMENT

The **while statement** allows you to *repeat* one or more actions while a condition remains `True`. Let's use a `while` statement to find the first power of 3 larger than 50:

[lick here to view code image](#)

```
In [1]: product = 3

In [2]: while product <= 50:
...:     product = product * 3
...:

In [3]: product
Out[3]: 81
```

Snippet [3] evaluates `product` to see its value, 81, which is the first power of 3 larger than 50.

Something in the `while` statement's suite must change `product`'s value, so the

condition eventually becomes `False`. Otherwise, an infinite loop occurs. In applications executed from a Terminal, Anaconda Command Prompt or shell, type *Ctrl* + *c* or *control* + *c* to terminate an infinite loop. IDEs typically have a toolbar button or menu option for stopping a program's execution.

3.6 FOR STATEMENT

The **for statement** allows you to *repeat* an action or several actions for each item in a **sequence** of items. For example, a string is a sequence of individual characters. Let's display 'Programming' with its characters separated by two spaces:

[lick here to view code image](#)

```
In [1]: for character in 'Programming':
...:     print(character, end='  ')
...:
P r o g r a m m i n g
```

The `for` statement executes as follows:

- Upon entering the statement, it assigns the 'P' in 'Programming' to the **target** variable between keywords `for` and `in`—in this case, `character`.
- Next, the statement in the suite executes, displaying `character`'s value followed by two spaces—we'll say more about this momentarily.
- After executing the suite, Python assigns to `character` the next item in the sequence (that is, the 'r' in 'Programming'), then executes the suite again.
- This continues while there are more items in the sequence to process. In this case, the statement terminates after displaying the letter 'g', followed by two spaces.

Using the target variable in the suite, as we did here to display its value, is common but not required.

Function `print`'s `end` Keyword Argument

The built-in function `print` displays its argument(s), then moves the cursor to the next line. You can change this behavior with the argument **`end`**, as in

```
print(character, end='  ')
```

which displays `character`'s value followed by two spaces. So, all the characters display horizontally on the *same* line. Python calls `end` a **keyword argument**, but `end` itself is not a Python keyword. Keyword arguments are sometimes called **named arguments**. The `end` keyword argument is optional. If you do not include it, `print` uses a newline (`'\n'`) by default. The *Style Guide for Python Code* recommends placing no spaces around a keyword argument's `=`.

Function `print`'s `sep` Keyword Argument

You can use the keyword argument `sep` (short for separator) to specify the string that appears *between* the items that `print` displays. When you do not specify this argument, `print` uses a space character by default. Let's display three numbers, each separated from the next by a comma and a space, rather than just a space:

[lick here to view code image](#)

```
In [2]: print(10, 20, 30, sep=', ')
10, 20, 30
```

To remove the default spaces, use `sep=''` (that is, an empty string).

3.6.1 Iterables, Lists and Iterators

The sequence to the right of the `for` statement's `in` keyword must be an **iterable**—that is, an object from which the `for` statement can take one item at a time until no more items remain. Python has other iterable sequence types besides strings. One of the most common is a **list**, which is a comma-separated collection of items enclosed in square brackets (`[` and `]`). The following code totals five integers in a list:

[lick here to view code image](#)

```
In [3]: total = 0

In [4]: for number in [2, -3, 0, 17, 9]:
...:     total = total + number
...:

In [5]: total
Out[5]: 25
```

Each sequence has an **iterator**. The `for` statement uses the iterator “behind the scenes” to get each consecutive item until there are no more to process. The iterator is like a bookmark—it always knows where it is in the sequence, so it can return the next item when it’s called upon to do so. We cover lists in detail in the “Sequences: Lists and Tuples” chapter. There, you’ll see that the order of the items in a list matters and that a list’s items are **mutable** (that is, modifiable).

3.6.2 Built-In `range` Function

Let’s use a `for` statement and the built-in **`range` function** to iterate precisely 10 times, displaying the values from 0 through 9:

[lick here to view code image](#)

```
In [6]: for counter in range(10):
...:     print(counter, end=' ')
...:
0 1 2 3 4 5 6 7 8 9
```

The function call `range(10)` creates an iterable object that represents a sequence of consecutive integers starting from 0 and continuing up to, but *not* including, the argument value (10)—in this case, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. The `for` statement exits when it finishes processing the last integer that `range` produces. Iterators and iterable objects are two of Python’s *functional-style programming* features. We’ll introduce more of these throughout the book.

Off-By-One Errors

A common type of off-by-one error occurs when you assume that `range`’s argument value is included in the generated sequence. For example, if you provide 9 as `range`’s argument when trying to produce the sequence 0 through 9, `range` generates only 0 through 8.

3.7 AUGMENTED ASSIGNMENTS

Augmented assignments abbreviate assignment expressions in which the same variable name appears on the left and right of the assignment’s `=`, as `total` does in:

[lick here to view code image](#)

```
for number in [1, 2, 3, 4, 5]:
```

```
total = total + number
```

Snippet [2] reimplements this using an **addition augmented assignment (+=) statement**:

[lick here to view code image](#)

```
In [1]: total = 0

In [2]: for number in [1, 2, 3, 4, 5]:
...:     total += number # add number to total
...:

In [3]: total
Out[3]: 15
```

The += expression in snippet [2] first adds number's value to the current total, then stores the new value in total. The table below shows sample augmented assignments:

Augmented assignment	Sample expression	Explanation	Assigns
<i>Assume: c = 3, d = 5, e = 4, f = 2, g = 9, h = 12</i>			
+=	c += 7	c = c + 7	10 to c
-=	d -= 4	d = d - 4	1 to d
*=	e *= 5	e = e * 5	20 to e
**=	f **= 3	f = f ** 3	8 to f
/=	g /= 2	g = g / 2	4.5 to g

```
//=                g //= 2                g = g // 2    4 to g

%=                h %= 9                h = h % 9    3 to h
```

3.8 SEQUENCE-CONTROLLED ITERATION; FORMATTED STRINGS

This section and the next solve two class-averaging problems. Consider the following requirements statement:

A class of ten students took a quiz. Their grades (integers in the range 0 – 100) are 98, 76, 71, 87, 83, 90, 57, 79, 82, 94. Determine the class average on the quiz.

The following script for solving this problem keeps a running total of the grades, calculates the average and displays the result. We placed the 10 grades in a list, but you could input the grades from a user at the keyboard (as we'll do in the next example) or read them from a file (as you'll see how to do in the “Files and Exceptions” chapter). We show how to read data from SQL and NoSQL databases in [chapter 16](#).

[lick here to view code image](#)

```
1 # class_average.py
2 """Class average program with sequence-controlled iteration."""
3
4 # initialization phase
5 total = 0 # sum of grades
6 grade_counter = 0
7 grades = [98, 76, 71, 87, 83, 90, 57, 79, 82, 94] # list of 10 grades
8
9 # processing phase
10 for grade in grades:
11     total += grade # add current grade to the running total
12     grade_counter += 1 # indicate that one more grade was process
13
14 # termination phase
15 average = total / grade_counter
16 print(f'Class average is {average}')
```

[lick here to view code image](#)

```
Class average is 81.7
```

Lines 5–6 create the variables `total` and `grade_counter` and initialize each to 0.
Line 7

[lick here to view code image](#)

```
grades = [98, 76, 71, 87, 83, 90, 57, 79, 82, 94] # list of 10 grades
```

creates the variable `grades` and initializes it with a list of 10 integer grades.

The `for` statement processes each grade in the list `grades`. Line 11 adds the current grade to the `total`. Then, line 12 adds 1 to the variable `grade_counter` to keep track of the number of grades processed so far. Iteration terminates when all 10 grades in the list have been processed. The *Style Guide for Python Code* recommends placing a blank line above and below each control statement (as in lines 8 and 13). When the `for` statement terminates, line 15 calculates the average and line 16 displays it. Later in this chapter, we use functional-style programming to calculate the average of a list's items more concisely.

Introduction to Formatted Strings

Line 16 uses the following simple **f-string** (short for **formatted string**) to format this script's result by inserting the value of `average` into a string:

[lick here to view code image](#)

```
f'Class average is {average}'
```

The letter `f` before the string's opening quote indicates it's an f-string. You specify where to insert values by using *placeholders* delimited by curly braces (`{` and `}`). The placeholder

```
{average}
```

converts the variable `average`'s value to a string representation, then replaces

{average} with that **replacement text**. Replacement-text expressions may contain values, variables or other expressions, such as calculations or function calls. In line 16, we could have used `total / grade_counter` in place of `average`, eliminating the need for line 15.

3.9 SENTINEL-CONTROLLED ITERATION

Let's generalize the class-average problem. Consider the following requirements statement:

Develop a class-averaging program that processes an arbitrary number of grades each time the program executes.

The requirements statement does not state what the grades are or how many there are, so we're going to have the user enter the grades. The program processes an arbitrary number of grades. The user enters grades one at a time until all the grades have been entered, then enters a *sentinel value* (also called a *signal value*, a *dummy value* or a *flag value*) to indicate that there are no more grades.

Implementing Sentinel-Controlled Iteration

The following script solves the class average problem with sentinel-controlled iteration. Notice that we test for the possibility of division by zero. If undetected, this would cause a fatal logic error. In the “Files and Exceptions” chapter, we write programs that recognize such exceptions and take appropriate actions.

[lick here to view code image](#)

```
1 # class_average_sentinel.py
2 """Class average program with sentinel-controlled iteration."""
3
4 # initialization phase
5 total = 0 # sum of grades
6 grade_counter = 0 # number of grades entered
7
8 # processing phase
9 grade = int(input('Enter grade, -1 to end: ')) # get one grade
10
11 while grade != -1:
12     total += grade
13     grade_counter += 1
14     grade = int(input('Enter grade, -1 to end: '))
15
16 # termination phase
17 if grade_counter != 0:
```

```
18     average = total / grade_counter
19     print(f'Class average is {average:.2f}')
20 else:
21     print('No grades were entered')
```

[lick here to view code image](#)

```
Enter grade, -1 to end: 97
Enter grade, -1 to end: 88
Enter grade, -1 to end: 72
Enter grade, -1 to end: -1
Class average is 85.67
```

Program Logic for Sentinel-Controlled Iteration

In sentinel-controlled iteration, the program reads the first value (line 9) before reaching the `while` statement. The value input in line 9 determines whether the program's flow of control should enter the `while`'s suite (lines 12–14). If the condition in line 11 is `False`, the user entered the sentinel value (`-1`), so the suite does not execute because the user did not enter any grades. If the condition is `True`, the suite executes, adding the `grade` value to the `total` and incrementing the `grade_counter`.

Next, line 14 inputs another grade from the user and the condition (line 11) is tested again, using the most recent `grade` entered by the user. The value of `grade` is always input immediately before the program tests the `while` condition, so we can determine whether the value just input is the sentinel before processing that value as a grade.

When the sentinel value is input, the loop terminates, and the program does not add `-1` to `total`. In a sentinel-controlled loop that performs user input, any prompts (lines 9 and 14) should remind the user of the sentinel value.

Formatting the Class Average with Two Decimal Places

This example formatted the class average with two digits to the right of the decimal point. In an f-string, you can optionally follow a replacement-text expression with a colon (`:`) and a **format specifier** that describes how to format the replacement text. The format specifier `.2f` (line 19) formats the `average` as a floating-point number (`f`) with two digits to the right of the decimal point (`.2`). In this example, the sum of the grades was 257, which, when divided by 3, yields 85.666666666.... Formatting the average with `.2f` *rounds* it to the hundredths position, producing the replacement

text 85.67. An average with only one digit to the right of the decimal point would be formatted with a **trailing zero** (e.g., 85.50). The chapter “Strings: A Deeper Look” discusses many more string-formatting features.

3.10 BUILT-IN FUNCTION `RANGE`: A DEEPER LOOK

Function `range` also has two- and three-argument versions. As you’ve seen, `range`’s one-argument version produces a sequence of consecutive integers from 0 up to, but not including, the argument’s value. Function `range`’s two-argument version produces a sequence of consecutive integers from its first argument’s value up to, but not including, the second argument’s value, as in:

[lick here to view code image](#)

```
In [1]: for number in range(5, 10):
...:     print(number, end=' ')
...:
5 6 7 8 9
```

Function `range`’s three-argument version produces a sequence of integers from its first argument’s value up to, but not including, the second argument’s value, *incrementing* by the third argument’s value, which is known as the **step**:

[lick here to view code image](#)

```
In [2]: for number in range(0, 10, 2):
...:     print(number, end=' ')
...:
0 2 4 6 8
```

If the third argument is negative, the sequence progresses from the first argument’s value *down* to, but not including the second argument’s value, *decrementing* by the third argument’s value, as in:

[lick here to view code image](#)

```
In [3]: for number in range(10, 0, -2):
...:     print(number, end=' ')
...:
10 8 6 4 2
```

3.11 USING TYPE DECIMAL FOR MONETARY AMOUNTS

In this section, we introduce `Decimal` capabilities for precise monetary calculations. If you're in banking or other fields that require “to-the-penny” accuracy, you should investigate `Decimal`'s capabilities in depth.

For most scientific and other mathematical applications that use numbers with decimal points, Python's built-in floating-point numbers work well. For example, when we speak of a “normal” body temperature of 98.6, we do not need to be precise to a large number of digits. When we view the temperature on a thermometer and read it as 98.6, the actual value may be 98.5999473210643. The point here is that calling this number 98.6 is adequate for most body-temperature applications.

Floating-point values are stored in binary format (we introduced binary in the first chapter and discuss it in depth in the online “Number Systems” appendix). Some floating-point values are represented only approximately when they're converted to binary. For example, consider the variable `amount` with the dollars-and-cents value `112.31`. If you display `amount`, it appears to have the exact value you assigned to it:

```
In [1]: amount = 112.31

In [2]: print(amount)
112.31
```

However, if you print `amount` with 20 digits of precision to the right of the decimal point, you can see that the actual floating-point value in memory is not exactly `112.31`—it's only an approximation:

[lick here to view code image](#)

```
In [3]: print(f'{amount:.20f}')
112.310000000000000227374
```

Many applications require *precise* representation of numbers with decimal points. Institutions like banks that deal with millions or even billions of transactions per day have to tie out their transactions “to the penny.” Floating-point numbers can represent some but not all monetary amounts with to-the-penny precision.

The **Python Standard Library**¹ provides many predefined capabilities you can use in your Python code to avoid “reinventing the wheel.” For monetary calculations and other applications that require precise representation and manipulation of numbers

with decimal points, the Python Standard Library provides type `Decimal`, which uses a special coding scheme to solve the problem of to-the-penny precision. That scheme requires additional memory to hold the numbers and additional processing time to perform calculations but provides the precision required for monetary calculations. Banks also have to deal with other issues such as using a *fair rounding algorithm* when they're calculating daily interest on accounts. Type `Decimal` offers such capabilities. ²

¹ <https://docs.python.org/3.7/library/index.html>.

² For more decimal module features, visit

<https://docs.python.org/3.7/library/decimal.html>.

Importing Type `Decimal` from the `decimal` Module

We've used several *built-in types*—`int` (for integers, like 10), `float` (for floating-point numbers, like 7.5) and `str` (for strings like 'Python'). The `Decimal` type is not built into Python. Rather, it's part of the Python Standard Library, which is divided into groups of related capabilities called **modules**. The `decimal` module defines type `Decimal` and its capabilities.

To use type `Decimal`, you must first **import** the entire `decimal` module, as in

```
import decimal
```

and refer to the `Decimal` type as `decimal.Decimal`, or you must indicate a specific capability to import using **from import**, as we do here:

[lick here to view code image](#)

```
In [4]: from decimal import Decimal
```

This imports only the type `Decimal` from the `decimal` module so that you can use it in your code. We'll discuss other `import` forms beginning in the next chapter.

Creating Decimals

You typically create a `Decimal` from a string:

[lick here to view code image](#)

```
In [5]: principal = Decimal('1000.00')

In [6]: principal
Out[6]: Decimal('1000.00')

In [7]: rate = Decimal('0.05')

In [8]: rate
Out[8]: Decimal('0.05')
```

We'll soon use these variables `principal` and `rate` in a compound-interest calculation.

Decimal Arithmetic

Decimals support the standard arithmetic operators `+`, `-`, `*`, `/`, `//`, `**` and `%`, as well as the corresponding augmented assignments:

```
In [9]: x = Decimal('10.5')

In [10]: y = Decimal('2')

In [11]: x + y
Out[11]: Decimal('12.5')

In [12]: x // y
Out[12]: Decimal('5')

In [13]: x += y

In [14]: x
Out[14]: Decimal('12.5')
```

You may perform arithmetic between `Decimal`s and integers, but *not* between `Decimal`s and floating-point numbers.

Compound-Interest Problem Requirements Statement

Let's compute compound interest using the `Decimal` type for *precise* monetary calculations. Consider the following requirements statement:

A person invests \$1000 in a savings account yielding 5% interest. Assuming that the person leaves all interest on deposit in the account, calculate and display the amount of money in the account at the end of each year for 10 years. Use the following formula for determining these amounts:

$$a = p(1 + r)^n$$

where

p is the original amount invested (i.e., the principal),

r is the annual interest rate,

n is the number of years and

a is the amount on deposit at the end of the n th year.

Calculating Compound Interest

To solve this problem, let's use variables `principal` and `rate` that we defined in snippets [5] and [7], and a `for` statement that performs the interest calculation for each of the 10 years the money remains on deposit. For each `year`, the loop displays a formatted string containing the year number and the amount on deposit at the end of that year:

[lick here to view code image](#)

```
In [15]: for year in range(1, 11):
...:     amount = principal * (1 + rate) ** year
...:     print(f'{year:>2}{amount:>10.2f}')
...:
1      1050.00
2      1102.50
3      1157.62
4      1215.51
5      1276.28
6      1340.10
7      1407.10
8      1477.46
9      1551.33
10     1628.89
```

The algebraic expression $(1 + r)^n$ from the requirements statement is written as

```
(1 + rate) ** year
```

where variable `rate` represents r and variable `year` represents n .

Formatting the Year and Amount on Deposit

The statement

[lick here to view code image](#)

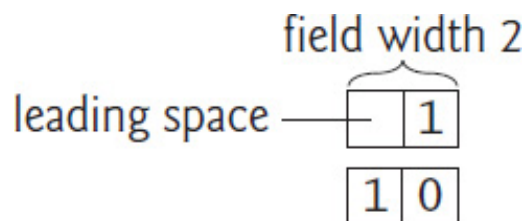
```
print(f'{year:>2}{amount:>10.2f}')
```

uses an f-string with two placeholders to format the loop's output.

The placeholder

```
{year:>2}
```

uses the format specifier `>2` to indicate that `year`'s value should be **right aligned (>)** in a field of width 2—the **field width** specifies the number of character positions to use when displaying the value. For the single-digit `year` values 1–9, the format specifier `>2` displays a space character followed by the value, thus right aligning the `years` in the first column. The following diagram shows the numbers 1 and 10 each formatted in a field width of 2:

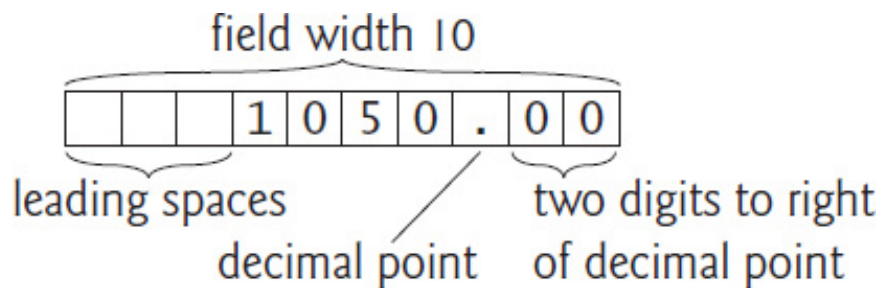


You can **left align** values with `<`.

The format specifier `10.2f` in the placeholder

```
{amount:>10.2f}
```

formats `amount` as a floating-point number (`f`) right aligned (`>`) in a field width of 10 with a decimal point and two digits to the right of the decimal point (`.2`). Formatting the `amounts` this way *aligns their decimal points vertically*, as is typical with monetary amounts. In the 10 character positions, the three rightmost characters are the number's decimal point followed by the two digits to its right. The remaining seven character positions are the leading spaces and the digits to the decimal point's left. In this example, all the dollar amounts have four digits to the left of the decimal point, so each number is formatted with three *leading spaces*. The following diagram shows the formatting for the value `1050.00`:



3.12 BREAK AND CONTINUE STATEMENTS

The `break` and `continue` statements alter a loop's flow of control. Executing a `break` statement in a `while` or `for` immediately exits that statement. In the following code, `range` produces the integer sequence 0–99, but the loop terminates when `number` is 10:

[lick here to view code image](#)

```
In [1]: for number in range(100):
...:     if number == 10:
...:         break
...:     print(number, end=' ')
...:
0 1 2 3 4 5 6 7 8 9
```

In a script, execution would continue with the next statement after the `for` loop. The `while` and `for` statements each have an optional `else` clause that executes only if the loop terminates normally—that is, not as a result of a `break`.

Executing a `continue` statement in a `while` or `for` loop skips the remainder of the loop's suite. In a `while`, the condition is then tested to determine whether the loop should continue executing. In a `for`, the loop processes the next item in the sequence (if any):

[lick here to view code image](#)

```
In [2]: for number in range(10):
...:     if number == 5:
...:         continue
...:     print(number, end=' ')
...:
0 1 2 3 4 6 7 8 9
```

3.13 BOOLEAN OPERATORS AND, OR AND NOT

The conditional operators `>`, `<`, `>=`, `<=`, `==` and `!=` can be used to form simple conditions such as `grade >= 60`. To form more complex conditions that combine simple conditions, use the `and`, `or` and `not` Boolean operators.

Boolean Operator `and`

To ensure that two conditions are *both* `True` before executing a control statement's suite, use the **Boolean `and` operator** to combine the conditions. The following code defines two variables, then tests a condition that's `True` if and only if *both* simple conditions are `True`—if either (or both) of the simple conditions is `False`, the entire `and` expression is `False`:

[lick here to view code image](#)

```
In [1]: gender = 'Female'

In [2]: age = 70

In [3]: if gender == 'Female' and age >= 65:
...:     print('Senior female')
...:
Senior female
```

The `if` statement has two simple conditions:

- `gender == 'Female'` determines whether a person is a female and
- `age >= 65` determines whether that person is a senior citizen.

The simple condition to the left of the `and` operator evaluates first because `==` has higher precedence than `and`. If necessary, the simple condition to the right of `and` evaluates next, because `>=` has higher precedence than `and`. (We'll discuss shortly why the right side of an `and` operator evaluates *only* if the left side is `True`.) The entire `if` statement condition is `True` if and only if *both* of the simple conditions are `True`. The combined condition can be made clearer by adding redundant parentheses

[lick here to view code image](#)

```
(gender == 'Female') and (age >= 65)
```

The table below summarizes the `and` operator by showing all four possible

combinations of `False` and `True` values for *expression1* and *expression2*—such tables are called *truth tables*:

<i>expression1</i>	<i>expression2</i>	<i>expression1</i> and <i>expression2</i>
<code>False</code>	<code>False</code>	<code>False</code>
<code>False</code>	<code>True</code>	<code>False</code>
<code>True</code>	<code>False</code>	<code>False</code>
<code>True</code>	<code>True</code>	<code>True</code>

Boolean Operator `or`

Use the **Boolean `or` operator** to test whether one *or* both of two conditions are `True`. The following code tests a condition that's `True` if either *or* both simple conditions are `True`—the entire condition is `False` only if *both* simple conditions are `False`:

[lick here to view code image](#)

```
In [4]: semester_average = 83

In [5]: final_exam = 95

In [6]: if semester_average >= 90 or final_exam >= 90:
...:     print('Student gets an A')
...:
Student gets an A
```

Snippet [6] also contains two simple conditions:

- `semester_average >= 90` determines whether a student's average was an A (90 or above) during the semester, and

- `final_exam >= 90` determines whether a student's final-exam grade was an A.

The truth table below summarizes the Boolean `or` operator. Operator `and` has higher precedence than `or`.

<i>expression1</i>	<i>expression2</i>	<i>expression1 or expression2</i>
False	False	False
False	True	True
True	False	True
True	True	True

Improving Performance with Short-Circuit Evaluation

Python stops evaluating an `and` expression as soon as it knows whether the entire condition is `False`. Similarly, Python stops evaluating an `or` expression as soon as it knows whether the entire condition is `True`. This is called *short-circuit evaluation*. So the condition

[lick here to view code image](#)

```
gender == 'Female' and age >= 65
```

stops evaluating immediately if `gender` is not equal to `'Female'` because the entire expression must be `False`. If `gender` is equal to `'Female'`, execution continues, because the entire expression will be `True` if the `age` is greater than or equal to 65.

Similarly, the condition

[lick here to view code image](#)

```
semester_average >= 90 or final_exam >= 90
```

stops evaluating immediately if `semester_average` is greater than or equal to 90 because the entire expression must be `True`. If `semester_average` is less than 90, execution continues, because the expression could still be `True` if the `final_exam` is greater than or equal to 90.

In expressions that use `and`, make the condition that's more likely to be `False` the leftmost condition. In `or` operator expressions, make the condition that's more likely to be `True` the leftmost condition. These techniques can reduce a program's execution time.

Boolean Operator `not`

The **Boolean operator `not`** “reverses” the meaning of a condition—`True` becomes `False` and `False` becomes `True`. This is a **unary operator**—it has only *one* operand. You place the `not` operator before a condition to choose a path of execution if the original condition (without the `not` operator) is `False`, such as in the following code:

[lick here to view code image](#)

```
In [7]: grade = 87

In [8]: if not grade == -1:
...:     print('The next grade is', grade)
...:
The next grade is 87
```

Often, you can avoid using `not` by expressing the condition in a more “natural” or convenient manner. For example, the preceding `if` statement can also be written as follows:

[lick here to view code image](#)

```
In [9]: if grade != -1:
...:     print('The next grade is', grade)
...:
The next grade is 87
```

he truth table below summarizes the `not` operator.

expression	<code>not</code> expression
False	True
True	False

The following table shows the precedence and grouping of the operators introduced so far, from top to bottom, in decreasing order of precedence.

Operators	Grouping
<code>()</code>	left to right
<code>**</code>	right to left
<code>*</code> <code>/</code> <code>//</code> <code>%</code>	left to right
<code>+</code> <code>-</code>	left to right
<code><</code> <code><=</code> <code>></code> <code>>=</code> <code>==</code> <code>!=</code>	left to right
<code>not</code>	left to right

and	left to right
or	left to right

3.14 INTRO TO DATA SCIENCE: MEASURES OF CENTRAL TENDENCY—MEAN, MEDIAN AND MODE

Here we continue our discussion of using statistics to analyze data with several additional descriptive statistics, including:

- **mean**—the *average value* in a set of values.
- **median**—the *middle value* when all the values are arranged in *sorted order*.
- **mode**—the *most frequently occurring value*.

These are **measures of central tendency**—each is a way of producing a single value that represents a “central” value in a set of values, i.e., a value which is in some sense typical of the others.

Let’s calculate the mean, median and mode on a list of integers. The following session creates a list called `grades`, then uses the built-in **sum** and **len** functions to calculate the mean “by hand”—`sum` calculates the total of the grades (397) and `len` returns the number of grades (5):

[lick here to view code image](#)

```
In [1]: grades = [85, 93, 45, 89, 85]

In [2]: sum(grades) / len(grades)
Out[2]: 79.4
```

The previous chapter mentioned the *descriptive statistics* `count` and `sum`—implemented in Python as the built-in functions `len` and `sum`. Like functions `min` and `max` (introduced in the preceding chapter), `sum` and `len` are both examples of functional-style programming **reductions**—they reduce a collection of values to a single value—the sum of those values and the number of values, respectively. In [section](#)

.8's class-average example, we could have deleted lines 10–15 of the script and replaced `average` in line 16 with snippet [2]'s calculation.

The Python Standard Library's **statistics module** provides functions for calculating the mean, median and mode—these, too, are reductions. To use these capabilities, first import the `statistics` module:

```
In [3]: import statistics
```

Then, you can access the module's functions with "`statistics.`" followed by the name of the function to call. The following calculates the `grades` list's mean, median and mode, using the `statistics` module's **mean**, **median** and **mode** functions:

[lick here to view code image](#)

```
In [4]: statistics.mean(grades)
Out[4]: 79.4

In [5]: statistics.median(grades)
Out[5]: 85

In [6]: statistics.mode(grades)
Out[6]: 85
```

Each function's argument must be an *iterable*—in this case, the list `grades`. To confirm that the median and mode are correct, you can use the built-in **sorted function** to get a copy of `grades` with its values arranged in increasing order:

```
In [7]: sorted(grades)
Out[7]: [45, 85, 85, 89, 93]
```

The `grades` list has an odd number of values (5), so `median` returns the *middle value* (85). If the list's number of values is even, `median` returns the *average* of the *two* middle values. Studying the sorted values, you can see that 85 is the mode because it occurs most frequently (twice). The `mode` function causes a `StatisticsError` for lists like

```
[85, 93, 45, 89, 85, 93]
```

in which there are two or more “most frequent” values. Such a set of values is said to be

imodal. Here, both 85 and 93 occur twice.

3.15 WRAP-UP

In this chapter, we discussed Python's control statements, including `if`, `if... else`, `if... elif... else`, `while`, `for`, `break` and `continue`. You saw that the `for` statement performs sequence-controlled iteration—it processes each item in an iterable, such as a range of integers, a string or a list. You used the built-in function `range` to generate sequences of integers from 0 up to, but not including, its argument, and to determine how many times a `for` statement iterates.

You used sentinel-controlled iteration with the `while` statement to create a loop that continues executing until a sentinel value is encountered. You used built-in function `range`'s two-argument version to generate sequences of integers from the first argument's value up to, but not including, the second argument's value. You also used the three-argument version in which the third argument indicated the step between integers in a range.

We introduced the `Decimal` type for precise monetary calculations and used it to calculate compound interest. You used f-strings and various format specifiers to create formatted output. We introduced the `break` and `continue` statements for altering the flow of control in loops. We discussed the Boolean operators `and`, `or` and `not` for creating conditions that combine simple conditions.

Finally, we continued our discussion of descriptive statistics by introducing measures of central tendency—mean, median and mode—and calculating them with functions from the Python Standard Library's `statistics` module.

In the next chapter, you'll create custom functions and use existing functions from Python's `math` and `random` modules. We show several predefined functional-programming reductions and you'll see additional functional-programming capabilities.

. Functions

Objectives

In this chapter, you'll

- Create custom functions.
- Import and use Python Standard Library modules, such as `random` and `math`, to reuse code and avoid “reinventing the wheel.”
- Pass data between functions.
- Generate a range of random numbers.
- See simulation techniques using random-number generation.
- Seed the random number generator to ensure reproducibility.
- Pack values into a tuple and unpack values from a tuple.
- Return multiple values from a function via a tuple.
- Understand how an identifier's scope determines where in your program you can use it.
- Create functions with default parameter values.
- Call functions with keyword arguments.
- Create functions that can receive any number of arguments.
- Use methods of an object.
- Write and use a recursive function.

- [.1 Introduction](#)
- [.2 Defining Functions](#)
- [.3 Functions with Multiple Parameters](#)
- [.4 Random-Number Generation](#)
- [.5 Case Study: A Game of Chance](#)
- [.6 Python Standard Library](#)
- [.7 `math` Module Functions](#)
- [.8 Using IPython Tab Completion for Discovery](#)
- [.9 Default Parameter Values](#)
- [.10 Keyword Arguments](#)
- [.11 Arbitrary Argument Lists](#)
- [.12 Methods: Functions That Belong to Objects](#)
- [.13 Scope Rules](#)
- [.14 `import`: A Deeper Look](#)
- [.15 Passing Arguments to Functions: A Deeper Look](#)
- [.16 Recursion](#)
- [.17 Functional-Style Programming](#)
- [.18 Intro to Data Science: Measures of Dispersion](#)
- [.19 Wrap-Up](#)

4.1 INTRODUCTION

In this chapter, we continue our discussion of Python fundamentals with custom functions and related topics. We'll use the Python Standard Library's `random` module and random-number generation to simulate rolling a six-sided die. We'll combine custom functions and random-number generation in a script that implements the dice game craps. In that example, we'll also introduce Python's tuple sequence type and use tuples to return more than one value from a function. We'll discuss seeding the random number generator to ensure reproducibility.

You'll import the Python Standard Library's `math` module, then use it to learn about IPython tab completion, which speeds your coding and discovery processes. You'll create functions with default parameter values, call functions with keyword arguments and define functions with arbitrary argument lists. We'll demonstrate calling methods of objects. We'll also discuss how an identifier's scope determines where in your program you can use it.

We'll take a deeper look at importing modules. You'll see that arguments are passed-by-reference to functions. We'll also demonstrate a recursive function and begin presenting Python's functional-style programming capabilities.

In the Intro to Data Science section, we'll continue our discussion of descriptive statistics by introducing measures of dispersion—variance and standard deviation—and calculating them with functions from the Python Standard Library's `statistics` module.

4.2 DEFINING FUNCTIONS

You've called many built-in functions (`int`, `float`, `print`, `input`, `type`, `sum`, `len`, `min` and `max`) and a few functions from the `statistics` module (`mean`, `median` and `mode`). Each performed a single, well-defined task. You'll often define and call *custom* functions. The following session defines a `square` function that calculates the square of its argument. Then it calls the function twice—once to square the `int` value 7 (producing the `int` value 49) and once to square the `float` value 2.5 (producing the `float` value 6.25):

[lick here to view code image](#)

```
In [1]: def square(number):
...:     """Calculate the square of number."""
...:     return number ** 2
...:
```

```
In [2]: square(7)
Out[2]: 49

In [3]: square(2.5)
Out[3]: 6.25
```

The statements defining the function in the first snippet are written only once, but may be called “to do their job” from many points throughout a program and as often as you like. Calling `square` with a non-numeric argument like `'hello'` causes a `TypeError` because the exponentiation operator (`**`) works only with numeric values.

Defining a Custom Function

A **function definition** (like `square` in snippet [1]) begins with the **`def` keyword**, followed by the function name (`square`), a set of parentheses and a colon (`:`). Like variable identifiers, by convention function names should begin with a lowercase letter and in multiword names underscores should separate each word.

The required parentheses contain the function’s **parameter list**—a comma-separated list of **parameters** representing the data that the function needs to perform its task. Function `square` has only one parameter named `number`—the value to be squared. If the parentheses are empty, the function does not use parameters to perform its task.

The indented lines after the colon (`:`) are the function’s **block**, which consists of an optional docstring followed by the statements that perform the function’s task. We’ll soon point out the difference between a function’s block and a control statement’s suite.

Specifying a Custom Function’s Docstring

The *Style Guide for Python Code* says that the first line in a function’s block should be a docstring that briefly explains the function’s purpose:

```
"""Calculate the square of number."""
```

To provide more detail, you can use a multiline docstring—the style guide recommends starting with a brief explanation, followed by a blank line and the additional details.

Returning a Result to a Function’s Caller

When a function finishes executing, it returns control to its caller—that is, the line of code that called the function. In `square`’s block, the **`return`** statement:

```
return number ** 2
```

first squares `number`, then terminates the function and gives the result back to the caller. In this example, the first caller is in snippet [2], so IPython displays the result in `Out[2]`. The second caller is in snippet [3], so IPython displays the result in `Out[3]`.

Function calls also can be embedded in expressions. The following code calls `square` first, then `print` displays the result:

[lick here to view code image](#)

```
In [4]: print('The square of 7 is', square(7))
The square of 7 is 49
```

There are two other ways to return control from a function to its caller:

- Executing a `return` statement without an expression terminates the function and *implicitly* returns the value **None** to the caller. The Python documentation states that `None` represents the absence of a value. `None` evaluates to `False` in conditions.
- When there's no `return` statement in a function, it *implicitly* returns the value `None` after executing the last statement in the function's block.

Local Variables

Though we did not define variables in `square`'s block, it is possible to do so. A function's parameters and variables defined in its block are all **local variables**—they can be used only inside the function and exist only while the function is executing. Trying to access a local variable outside its function's block causes a `NameError`, indicating that the variable is not defined.

Accessing a Function's Docstring via IPython's Help Mechanism

IPython can help you learn about the modules and functions you intend to use in your code, as well as IPython itself. For example, to view a function's docstring to learn how to use the function, type the function's name followed by a **question mark (?)**:

[lick here to view code image](#)

```
In [5]: square?
Signature: square(number)
Docstring: Calculate the square of number.
File:      ~/Documents/examples/ch04/<ipython-input-1-7268c8ff93a9>
Type:      function
```

For our `square` function, the information displayed includes:

- The function’s name and parameter list—known as its **signature**.
- The function’s docstring.
- The name of the file containing the function’s definition. For a function in an interactive session, this line shows information for the snippet that defined the function—the 1 in "<ipython-input-1-7268c8ff93a9>" means snippet [1].
- The type of the item for which you accessed IPython’s help mechanism—in this case, a function.

If the function’s source code is accessible from IPython—such as a function defined in the current session or imported into the session from a `.py` file—you can use `??` to display the function’s full source-code definition:

[lick here to view code image](#)

```
In [6]: square??
Signature: square(number)
Source:
def square(number):
    """Calculate the square of number."""
    return number ** 2
File:      ~/Documents/examples/ch04/<ipython-input-1-7268c8ff93a9>
Type:      function
```

If the source code is not accessible from IPython, `??` simply shows the docstring.

If the docstring fits in the window, IPython displays the next `In []` prompt. If a docstring is too long to fit, IPython indicates that there’s more by displaying a colon (`:`) at the bottom of the window—press the *Space* key to display the next screen. You can navigate backwards and forwards through the docstring with the up and down arrow keys, respectively. IPython displays `(END)` at the end of the docstring. Press *q* (for “quit”) at any `:` or the `(END)` prompt to return to the next `In []` prompt. To get a

sense of IPython's features, type `?` at any `In []` prompt, press *Enter*, then read the help documentation overview.

4.3 FUNCTIONS WITH MULTIPLE PARAMETERS

Let's define a `maximum` function that determines and returns the largest of three values—the following session calls the function three times with integers, floating-point numbers and strings, respectively.

[lick here to view code image](#)

```
In [1]: def maximum(value1, value2, value3):
...:     """Return the maximum of three values."""
...:     max_value = value1
...:     if value2 > max_value:
...:         max_value = value2
...:     if value3 > max_value:
...:         max_value = value3
...:     return max_value
...:

In [2]: maximum(12, 27, 36)
Out[2]: 36

In [3]: maximum(12.3, 45.6, 9.7)
Out[3]: 45.6

In [4]: maximum('yellow', 'red', 'orange')
Out[4]: 'yellow'
```

We did not place blank lines above and below the `if` statements, because pressing `return` on a blank line in interactive mode completes the function's definition.

You also may call `maximum` with mixed types, such as `ints` and `floats`:

```
In [5]: maximum(13.5, -3, 7)
Out[5]: 13.5
```

The call `maximum(13.5, 'hello', 7)` results in `TypeError` because strings and numbers cannot be compared to one another with the greater-than (`>`) operator.

Function `maximum`'s Definition

Function `maximum` specifies three parameters in a comma-separated list. Snippet [2]'s

arguments 12, 27 and 36 are assigned to the parameters `value1`, `value2` and `value3`, respectively.

To determine the largest value, we process one value at a time:

- Initially, we assume that `value1` contains the largest value, so we assign it to the local variable `max_value`. Of course, it's possible that `value2` or `value3` contains the actual largest value, so we still must compare each of these with `max_value`.
- The first `if` statement then tests `value2 > max_value`, and if this condition is `True` assigns `value2` to `max_value`.
- The second `if` statement then tests `value3 > max_value`, and if this condition is `True` assigns `value3` to `max_value`.

Now, `max_value` contains the largest value, so we return it. When control returns to the caller, the parameters `value1`, `value2` and `value3` and the variable `max_value` in the function's block—which are all *local variables*—no longer exist.

Python's Built-In `max` and `min` Functions

For many common tasks, the capabilities you need already exist in Python. For example, built-in `max` and `min` functions know how to determine the largest and smallest of their two or more arguments, respectively:

[lick here to view code image](#)

```
In [6]: max('yellow', 'red', 'orange', 'blue', 'green')
Out[6]: 'yellow'

In [7]: min(15, 9, 27, 14)
Out[7]: 9
```

Each of these functions also can receive an iterable argument, such as a list or a string. Using built-in functions or functions from the Python Standard Library's modules rather than writing your own can reduce development time and increase program reliability, portability and performance. For a list of Python's built-in functions and modules, see

<https://docs.python.org/3/library/index.html>

4.4 RANDOM-NUMBER GENERATION

We now take a brief diversion into a popular type of programming application—simulation and game playing. You can introduce the **element of chance** via the Python Standard Library’s **random module**.

Rolling a Six-Sided Die

Let’s produce 10 random integers in the range 1–6 to simulate rolling a six-sided die:

[lick here to view code image](#)

```
In [1]: import random

In [2]: for roll in range(10):
...:     print(random.randrange(1, 7), end=' ')
...:
4 2 5 5 4 6 4 6 1 5
```

First, we import `random` so we can use the module’s capabilities. The **randrange** function generates an integer from the first argument value up to, but *not* including, the second argument value. Let’s use the up arrow key to recall the `for` statement, then press *Enter* to re-execute it. Notice that *different* values are displayed:

[lick here to view code image](#)

```
In [3]: for roll in range(10):
...:     print(random.randrange(1, 7), end=' ')
...:
4 5 4 5 1 4 1 4 6 5
```

Sometimes, you may want to guarantee **reproducibility** of a random sequence—for debugging, for example. At the end of this section, we’ll use the `random` module’s `seed` function to do this.

Rolling a Six-Sided Die 6,000,000 Times

If `randrange` truly produces integers at random, every number in its range has an equal **probability** (or *chance* or *likelihood*) of being returned each time we call it. To show that the die faces 1–6 occur with equal likelihood, the following script simulates 6,000,000 die rolls. When you run the script, each die face should occur approximately 1,000,000 times, as in the sample output.

[lick here to view code image](#)

```
1 # fig04_01.py
2 """Roll a six-sided die 6,000,000 times."""
3 import random
4
5 # face frequency counters
6 frequency1 = 0
7 frequency2 = 0
8 frequency3 = 0
9 frequency4 = 0
10 frequency5 = 0
11 frequency6 = 0
12
13 # 6,000,000 die rolls
14 for roll in range(6_000_000): # note underscore separators
15     face = random.randrange(1, 7)
16
17     # increment appropriate face counter
18     if face == 1:
19         frequency1 += 1
20     elif face == 2:
21         frequency2 += 1
22     elif face == 3:
23         frequency3 += 1
24     elif face == 4:
25         frequency4 += 1
26     elif face == 5:
27         frequency5 += 1
28     elif face == 6:
29         frequency6 += 1
30
31 print(f'Face{"Frequency":>13}')
32 print(f'{1:>4}{frequency1:>13}')
33 print(f'{2:>4}{frequency2:>13}')
34 print(f'{3:>4}{frequency3:>13}')
35 print(f'{4:>4}{frequency4:>13}')
36 print(f'{5:>4}{frequency5:>13}')
37 print(f'{6:>4}{frequency6:>13}')
```

[lick here to view code image](#)

Face	Frequency
1	998686
2	1001481
3	999900
4	1000453
5	999953
6	999527

The script uses *nested* control statements (an `if elif` statement nested in the `for` statement) to determine the number of times each die face appears. The `for` statement iterates 6,000,000 times. We used Python's underscore (`_`) digit separator to make the value `6000000` more readable. The expression `range(6,000,000)` would be incorrect. Commas separate arguments in function calls, so Python would treat `range(6,000,000)` as a call to `range` with the *three* arguments 6, 0 and 0.

For each die roll, the script adds 1 to the appropriate counter variable. Run the program, and observe the results. This program might take a few seconds to complete execution. As you'll see, each execution produces *different* results. Note that we did not provide an `else` clause in the `if elif` statement.

Seeding the Random-Number Generator for Reproducibility

Function `randrange` actually generates **pseudorandom numbers**, based on an internal calculation that begins with a numeric value known as a **seed**. Repeatedly calling `randrange` produces a sequence of numbers that *appear* to be random, because each time you start a new interactive session or execute a script that uses the `random` module's functions, Python internally uses a *different* seed value.¹ When you're debugging logic errors in programs that use randomly generated data, it can be helpful to use the *same* sequence of random numbers until you've eliminated the logic errors, before testing the program with other values. To do this, you can use the `random` module's **seed** function to **seed the random-number generator** yourself—this forces `randrange` to begin calculating its pseudorandom number sequence from the seed you specify. In the following session, snippets [5] and [8] produce the same results, because snippets [4] and [7] use the same seed (32):

¹ According to the documentation, Python bases the seed value on the system clock or an operating-system-dependent randomness source. For applications requiring secure random numbers, such as cryptography, the documentation recommends using the `secrets` module, rather than the `random` module.

[lick here to view code image](#)

```
In [4]: random.seed(32)

In [5]: for roll in range(10):
...:     print(random.randrange(1, 7), end=' ')
...:
1 2 2 3 6 2 4 1 6 1
In [6]: for roll in range(10):
...:     print(random.randrange(1, 7), end=' ')
```

```

...:
1 3 5 3 1 5 6 4 3 5
In [7]: random.seed(32)

In [8]: for roll in range(10):
...:     print(random.randrange(1, 7), end=' ')
...:
1 2 2 3 6 2 4 1 6 1

```

Snippet [6] generates *different* values because it simply continues the pseudorandom number sequence that began in snippet [5].

4.5 CASE STUDY: A GAME OF CHANCE

In this section, we simulate the popular dice game known as “craps.” Here is the requirements statement:

You roll two six-sided dice, each with faces containing one, two, three, four, five and six spots, respectively. When the dice come to rest, the sum of the spots on the two upward faces is calculated. If the sum is 7 or 11 on the first roll, you win. If the sum is 2, 3 or 12 on the first roll (called “craps”), you lose (i.e., the “house” wins). If the sum is 4, 5, 6, 8, 9 or 10 on the first roll, that sum becomes your “point.” To win, you must continue rolling the dice until you “make your point” (i.e., roll that same point value). You lose by rolling a 7 before making your point.

The following script simulates the game and shows several sample executions, illustrating winning on the first roll, losing on the first roll, winning on a subsequent roll and losing on a subsequent roll.

[lick here to view code image](#)

```

1 # fig04_02.py
2 """Simulating the dice game    Craps."""
3 import random
4
5 def roll_dice():
6     """Roll two dice and return their face    values as a tuple."""
7     die1 =    random.randrange(1, 7)
8     die2 =    random.randrange(1, 7)
9     return (die1, die2)    # pack die    face values into a tuple
10
11 def display_dice(dice):
12     """Display one roll of the two    dice."""
13     die1, die2    = dice    # unpack the tuple into variables die1 and
14     print(f'Player rolled {die1} +    {die2} = {sum(dice)}')

```

```

15
16 die_values = roll_dice() # first roll
17 display_dice(die_values)
18
19 # determine game status and point, based on first roll
20 sum_of_dice = sum(die_values)
21
22 if sum_of_dice in (7, 11): # win
23     game_status = 'WON'
24 elif sum_of_dice in (2, 3, 12): # lose
25     game_status = 'LOST'
26 else: # remember point
27     game_status = 'CONTINUE'
28     my_point = sum_of_dice
29     print('Point is', my_point)
30
31 # continue rolling until player wins or loses
32 while game_status == 'CONTINUE':
33     die_values = roll_dice()
34     display_dice(die_values)
35     sum_of_dice = sum(die_values)
36
37     if sum_of_dice == my_point: # win by making point
38         game_status = 'WON'
39     elif sum_of_dice == 7: # lose by rolling 7
40         game_status = 'LOST'
41
42 # display "wins" or "loses" message
43 if game_status == 'WON':
44     print('Player wins')
45 else:
46     print('Player loses')

```

[lick here to view code image](#)

```

Player rolled 2 + 5 = 7
Player wins

```

[lick here to view code image](#)

```

Player rolled 1 + 2 = 3
Player loses

```

[lick here to view code image](#)

```
Player rolled 5 + 4 = 9
Point is 9
Player rolled 4 + 4 = 8
Player rolled 2 + 3 = 5
Player rolled 5 + 4 = 9
Player wins
```

[lick here to view code image](#)

```
Player rolled 1 + 5 = 6
Point is 6
Player rolled 1 + 6 = 7
Player loses
```

Function `roll_dice`—Returning Multiple Values Via a Tuple

Function `roll_dice` (lines 5–9) simulates rolling two dice on each roll. The function is defined once, then called from several places in the program (lines 16 and 33). The empty parameter list indicates that `roll_dice` does not require arguments to perform its task.

The built-in and custom functions you’ve called so far each return one value.

Sometimes it’s useful to return more than one value, as in `roll_dice`, which returns both die values (line 9) as a **tuple**—an **immutable** (that is, unmodifiable) sequences of values. To create a tuple, separate its values with commas, as in line 9:

```
(die1, die2)
```

This is known as **packing a tuple**. The parentheses are optional, but we recommend using them for clarity. We discuss tuples in depth in the next chapter.

Function `display_dice`

To use a tuple’s values, you can assign them to a comma-separated list of variables, which **unpacks** the tuple. To display each roll of the dice, the function `display_dice` (defined in lines 11–14 and called in lines 17 and 34) unpacks the tuple argument it receives (line 13). The number of variables to the left of `=` must match the number of elements in the tuple; otherwise, a `ValueError` occurs. Line 14 prints a formatted string containing both die values and their sum. We calculate the sum of the dice by

passing the tuple to the built-in `sum` function—like a list, a tuple is a sequence.

Note that functions `roll_dice` and `display_dice` each begin their blocks with a docstring that states what the function does. Also, both functions contain local variables `die1` and `die2`. These variables do not “collide,” because they belong to different functions’ blocks. Each local variable is accessible only in the block that defined it.

First Roll

When the script begins executing, lines 16–17 roll the dice and display the results. Line 20 calculates the sum of the dice for use in lines 22–29. You can win or lose on the first roll or any subsequent roll. The variable `game_status` keeps track of the win/loss status.

The **in operator** in line 22

```
sum_of_dice in (7, 11)
```

tests whether the tuple `(7, 11)` contains `sum_of_dice`’s value. If this condition is `True`, you rolled a 7 or an 11. In this case, you won on the first roll, so the script sets `game_status` to `'WON'`. The operator’s right operand can be any iterable. There’s also a **not in** operator to determine whether a value is *not* in an iterable. The preceding concise condition is equivalent to

[lick here to view code image](#)

```
(sum_of_dice == 7) or (sum_of_dice == 11)
```

Similarly, the condition in line 24

```
sum_of_dice in (2, 3, 12)
```

tests whether the tuple `(2, 3, 12)` contains `sum_of_dice`’s value. If so, you lost on the first roll, so the script sets `game_status` to `'LOST'`.

For any other sum of the dice (4, 5, 6, 8, 9 or 10):

- line 27 sets `game_status` to `'CONTINUE'` so you can continue rolling

- line 28 stores the sum of the dice in `my_point` to keep track of what you must roll to win and
- line 29 displays `my_point`.

Subsequent Rolls

If `game_status` is equal to `'CONTINUE'` (line 32), you did not win or lose, so the `while` statement's suite (lines 33–40) executes. Each loop iteration calls `roll_dice`, displays the die values and calculates their sum. If `sum_of_dice` is equal to `my_point` (line 37) or 7 (line 39), the script sets `game_status` to `'WON'` or `'LOST'`, respectively, and the loop terminates. Otherwise, the `while` loop continues executing with the next roll.

Displaying the Final Results

When the loop terminates, the script proceeds to the `if else` statement (lines 43–46), which prints `'Player wins'` if `game_status` is `'WON'`, or `'Player loses'` otherwise.

4.6 PYTHON STANDARD LIBRARY

Typically, you write Python programs by combining functions and classes (that is, custom types) that you create with preexisting functions and classes defined in modules, such as those in the Python Standard Library and other libraries. A key programming goal is to avoid “reinventing the wheel.”

A module is a file that groups related functions, data and classes. The type `Decimal` from the Python Standard Library's `decimal` module is actually a class. We introduced classes briefly in [Chapter 1](#) and discuss them in detail in the “Object-Oriented Programming” chapter. A **package** groups related modules. In this book, you'll work with many preexisting modules and packages, and you'll create your own modules—in fact, every Python source-code (`.py`) file you create is a module. Creating packages is beyond this book's scope. They're typically used to organize a large library's functionality into smaller subsets that are easier to maintain and can be imported separately for convenience. For example, the `matplotlib` visualization library that we use in [Section 5.17](#) has extensive functionality (its documentation is over 2300 pages), so we'll import only the subsets we need in our examples (`pyplot` and `animation`).

The Python Standard Library is provided with the core Python language. Its packages and modules contain capabilities for a wide variety of everyday programming tasks. ²

ou can see a complete list of the standard library modules at

² The Python Tutorial refers to this as the batteries included approach.

<https://docs.python.org/3/library/>

You’ve already used capabilities from the `decimal`, `statistics` and `random` modules. In the next section, you’ll use mathematics capabilities from the `math` module. You’ll see many other Python Standard Library modules throughout the book’s examples, including many of those in the following table:

Some popular Python Standard Library modules	
	<code>math</code> —Common math constants and operations.
<code>collections</code> —Data structures beyond lists, tuples, dictionaries and sets.	<code>os</code> —Interacting with the operating system.
Cryptography modules—Encrypting data for secure transmission.	<code>profile</code> , <code>pstats</code> , <code>timeit</code> —Performance analysis.
<code>csv</code> —Processing comma-separated value files (like those in Excel).	<code>random</code> —Pseudorandom numbers.
<code>datetime</code> —Date and time manipulations. Also modules <code>time</code> and <code>calendar</code> .	<code>re</code> —Regular expressions for pattern matching.
<code>decimal</code> —Fixed-point and floating-point arithmetic, including monetary calculations.	<code>sqlite3</code> —SQLite relational database access.
<code>doctest</code> —Embed validation tests and expected results in docstrings for simple unit testing.	<code>statistics</code> —Mathematical statistics functions such as <code>mean</code> , <code>median</code> , <code>mode</code> and <code>variance</code> .
	<code>string</code> —String processing.
	<code>sys</code> —Command-line argument

`gettext` and `locale`—Internationalization and localization modules.

`json`—JavaScript Object Notation (JSON) processing used with web services and NoSQL document databases.

processing; standard input, standard output and standard error streams.

`tkinter`—Graphical user interfaces (GUIs) and canvas-based graphics.

`turtle`—Turtle graphics.

`webbrowser`—For conveniently displaying web pages in Python apps.

4.7 MATH MODULE FUNCTIONS

The **math module** defines functions for performing various common mathematical calculations. Recall from the previous chapter that an `import` statement of the following form enables you to use a module's definitions via the module's name and a dot (`.`):

```
In [1]: import math
```

For example, the following snippet calculates the square root of 900 by calling the `math` module's **`sqrt` function**, which returns its result as a `float` value:

```
In [2]: math.sqrt(900)
Out[2]: 30.0
```

Similarly, the following snippet calculates the absolute value of `-10` by calling the `math` module's **`fabs` function**, which returns its result as a `float` value:

```
In [3]: math.fabs(-10)
Out[3]: 10.0
```

Some `math` module functions are summarized below—you can view the complete list at

Function	Description	Example
<code>ceil(x)</code>	Rounds x to the smallest integer not less than x	<code>ceil(9.2)</code> is 10.0 <code>ceil(-9.8)</code> is -9.0
<code>floor(x)</code>	Rounds x to the largest integer not greater than x	<code>floor(9.2)</code> is 9.0 <code>floor(-9.8)</code> is -10.0
<code>sin(x)</code>	Trigonometric sine of x (x in radians)	<code>sin(0.0)</code> is 0.0
<code>cos(x)</code>	Trigonometric cosine of x (x in radians)	<code>cos(0.0)</code> is 1.0
<code>tan(x)</code>	Trigonometric tangent of x (x in radians)	<code>tan(0.0)</code> is 0.0
<code>exp(x)</code>	Exponential function e^x	<code>exp(1.0)</code> is 2.718282 <code>exp(2.0)</code> is 7.389056
		<code>log(2.718282)</code> is 1.0

<code>log(x)</code>	Natural logarithm of x (base e)	<code>log(7.389056)</code> is 2.0
<code>log10(x)</code>	Logarithm of x (base 10)	<code>log10(10.0)</code> is 1.0 <code>log10(100.0)</code> is 2.0
<code>pow(x, y)</code>	x raised to power y (x^y)	<code>pow(2.0, 7.0)</code> is 128.0 <code>pow(9.0, .5)</code> is 3.0
<code>sqrt(x)</code>	square root of x	<code>sqrt(900.0)</code> is 30.0 <code>sqrt(9.0)</code> is 3.0
<code>fabs(x)</code>	Absolute value of x —always returns a float. Python also has the built-in function <code>abs</code> , which returns an <code>int</code> or a <code>float</code> , based on its argument.	<code>fabs(5.1)</code> is 5.1 <code>fabs(-5.1)</code> is 5.1
<code>fmod(x, y)</code>	Remainder of x/y as a floating-point number	<code>fmod(9.8, 4.0)</code> is 1.8

4.8 USING IPYTHON TAB COMPLETION FOR DISCOVERY

You can view a module's documentation in IPython interactive mode via **tab completion**—a **discovery** feature that speeds your coding and discovery processes. After you type a portion of an identifier and press *Tab*, IPython completes the identifier for you or provides a list of identifiers that begin with what you've typed so far. This may vary based on your operating system platform and what you have imported into your IPython session:

[lick here to view code image](#)

```
In [1]: import math

In [2]: ma<Tab>
        map          %macro      %%markdown
        math         %magic      %matplotlib
        max()        %man
```

You can scroll through the identifiers with the up and down arrow keys. As you do, IPython highlights an identifier and shows it to the right of the `In []` prompt.

Viewing Identifiers in a Module

To view a list of identifiers defined in a module, type the module's name and a dot (`.`), then press *Tab*:

[lick here to view code image](#)

```
In [3]: math.<Tab>
        acos()      atan()      copysign()  e          expm1()
        acosh()     atan2()     cos()      erf()      fabs()
        asin()      atanh()     cosh()     erfc()     factorial() >
        asinh()     ceil()      degrees()  exp()      floor()
```

If there are more identifiers to display than are currently shown, IPython displays the `>` symbol (on some platforms) at the right edge, in this case to the right of `factorial()`. You can use the up and down arrow keys to scroll through the list. In the list of identifiers:

- Those followed by parentheses are functions (or methods, as you'll see later).
- Single-word identifiers (such as `Employee`) that begin with an uppercase letter and

multiword identifiers in which each word begins with an uppercase letter (such as `CommissionEmployee`) represent class names (there are none in the preceding list). This naming convention, which the *Style Guide for Python Code* recommends, is known as **CamelCase** because the uppercase letters stand out like a camel's humps.

- Lowercase identifiers without parentheses, such as `pi` (not shown in the preceding list) and `e`, are variables. The identifier `pi` evaluates to `3.141592653589793`, and the identifier `e` evaluates to `2.718281828459045`. In the `math` module, `pi` and `e` represent the mathematical constants π and e , respectively.

Python does not have *constants*, although many objects in Python are immutable (nonmodifiable). So even though `pi` and `e` are real-world constants, *you must not assign new values to them*, because that would change their values. To help distinguish constants from other variables, the style guide recommends naming your custom constants with all capital letters.

Using the Currently Highlighted Function

As you navigate through the identifiers, if you wish to use a currently highlighted function, simply start typing its arguments in parentheses. IPython then hides the autocompletion list. If you need more information about the currently highlighted item, you can view its docstring by typing a question mark (?) following the name and pressing *Enter* to view the help documentation. The following shows the `fabs` function's docstring:

[lick here to view code image](#)

```
In [4]: math.fabs?
Docstring:
fabs(x)

Return the absolute value of the float x.
Type:      builtin_function_or_method
```

The `builtin_function_or_method` shown above indicates that `fabs` is part of a Python Standard Library module. Such modules are considered to be built into Python. In this case, `fabs` is a built-in function from the `math` module.

4.9 DEFAULT PARAMETER VALUES

When defining a function, you can specify that a parameter has a **default parameter value**. When calling the function, if you omit the argument for a parameter with a default parameter value, the default value for that parameter is automatically passed. Let's define a function `rectangle_area` with default parameter values:

[lick here to view code image](#)

```
In [1]: def    rectangle_area(length=2, width=3):  
...:     """Return    a rectangle's area."""  
...:     return length *    width  
...:
```

You specify a default parameter value by following a parameter's name with an `=` and a value—in this case, the default parameter values are 2 and 3 for `length` and `width`, respectively. Any parameters with default parameter values must appear in the parameter list to the *right* of parameters that do not have defaults.

The following call to `rectangle_area` has no arguments, so IPython uses both default parameter values as if you had called `rectangle_area(2, 3)`:

```
In [2]: rectangle_area()  
Out[2]: 6
```

The following call to `rectangle_area` has only one argument. Arguments are assigned to parameters from left to right, so 10 is used as the `length`. The interpreter passes the default parameter value 3 for the `width` as if you had called `rectangle_area(10, 3)`:

```
In [3]: rectangle_area(10)  
Out[3]: 30
```

The following call to `rectangle_area` has arguments for both `length` and `width`, so IPython- ignores the default parameter values:

```
In [4]: rectangle_area(10, 5)  
Out[4]: 50
```

4.10 KEYWORD ARGUMENTS

When calling functions, you can use **keyword arguments** to pass arguments in *any* order. To demonstrate keyword arguments, we redefine the `rectangle_area` function—this time without default parameter values:

[lick here to view code image](#)

```
In [1]: def    rectangle_area(length, width):  
...:     """Return    a rectangle's area."""  
...:     return length *    width  
...:
```

Each keyword *argument in a call* has the form *parametername=value*. The following call shows that the order of keyword arguments does not matter—they do not need to match the corresponding parameters' positions in the function definition:

[lick here to view code image](#)

```
In [2]: rectangle_area(width=5,    length=10)  
Out[3]: 50
```

In each function call, you must place keyword arguments *after* a function's positional arguments—that is, any arguments for which you do not specify the parameter name. Such arguments are assigned to the function's parameters left-to-right, based on the argument's positions in the argument list. Keyword arguments are also helpful for improving the readability of function calls, especially for functions with many arguments.

4.11 ARBITRARY ARGUMENT LISTS

Functions with **arbitrary argument lists**, such as built-in functions `min` and `max`, can receive *any* number of arguments. Consider the following `min` call:

```
min(88, 75, 96, 55, 83)
```

The function's documentation states that `min` has two *required* parameters (named `arg1` and `arg2`) and an optional third parameter of the form ***args**, indicating that the function can receive any number of additional arguments. The `*` before the parameter name tells Python to pack any remaining arguments into a tuple that's passed to the `args` parameter. In the call above, parameter `arg1` receives 88,

parameter `arg2` receives 75 and parameter `args` receives the tuple `(96, 55, 83)`.

Defining a Function with an Arbitrary Argument List

Let's define an `average` function that can receive any number of arguments:

[lick here to view code image](#)

```
In [1]: def average(*args):  
...:     return sum(args) / len(args)  
...:
```

The parameter name `args` is used by convention, but you may use any identifier. If the function has multiple parameters, the `*args` parameter must be the *rightmost* parameter.

Now, let's call `average` several times with arbitrary argument lists of different lengths:

[lick here to view code image](#)

```
In [2]: average(5, 10)  
Out[2]: 7.5  
  
In [3]: average(5, 10, 15)  
Out[3]: 10.0  
  
In [4]: average(5, 10, 15, 20)  
Out[4]: 12.5
```

To calculate the average, divide the sum of the `args` tuple's elements (returned by built-in function `sum`) by the tuple's number of elements (returned by built-in function `len`). Note in our `average` definition that if the length of `args` is 0, a `ZeroDivisionError` occurs. In the next chapter, you'll see how to access a tuple's elements without unpacking them.

Passing an Iterable's Individual Elements as Function Arguments

You can unpack a tuple's, list's or other iterable's elements to pass them as individual function arguments. The *** operator**, when applied to an iterable argument in a function call, unpacks its elements. The following code creates a five-element `grades` list, then uses the expression `*grades` to unpack its elements as `average`'s arguments: